

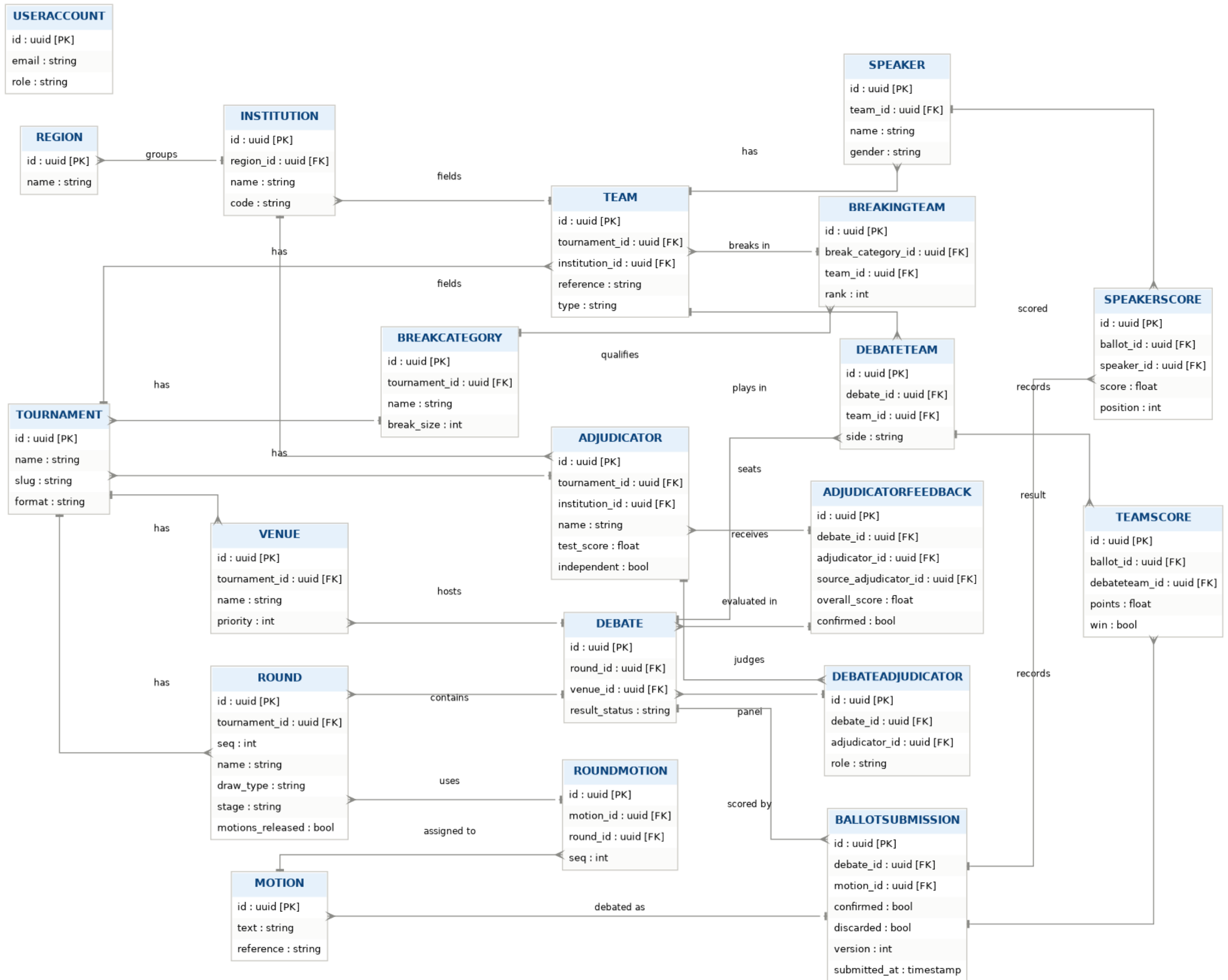
Debate tab app

Database schema

Relational (SQL) entity-relationship diagram, plus a Firestore alternative

Based on the Tabbycat tournament tabulation system

Relational schema (SQL)



Arrow reads tee → crow: one tournament has many rounds, one round contains many debates, and so on.

Firestore schema (NoSQL alternative)

```
tournaments/{tournamentId}
  name, slug, format: "BP" | "AP"

rounds/{roundId}
  seq, name, drawType, stage, motionsReleased: bool
  motionRefs: [motionId, ...]

debates/{debateId}
  venueRef, resultStatus
  teams: [
    { teamRef, side, points, win }
  ]
  panel: [
    { adjudicatorRef, role }
  ]

ballots/{ballotId}
  motionRef, confirmed, discarded, version, submittedAt
  speakerScores: [
    { speakerRef, score, position }
  ]

breakCategories/{categoryId}
  name, breakSize
  breakingTeams: [ { teamRef, rank } ]

teams/{teamId}
  tournamentRef, institutionRef, reference, type
  speakers: [ { name, gender } ]

institutions/{institutionId}
  regionRef, name, code

adjudicators/{adjudicatorId}
  tournamentRef, institutionRef, name, testScore, independent

feedback/{feedbackId}
  debateRef, sourceAdjudicatorRef, overallScore, confirmed

venues/{venueId}
  tournamentRef, name, priority

motions/{motionId}
  text, reference

users/{userId}
  email, role
```

Trade-offs worth flagging

- Speakers embedded in teams — fine unless speaker-level standings or cross-team speaker search matters, in which case promote to a top-level collection.
- The teams array on each debate replaces the join table and score row — cheap to read in one call, but updating a score means rewriting part of the debate document.
- Ballots stay a subcollection, not embedded — each submission should be its own document so the version history survives edits and re-confirmations.